PROJECT REPORT ARTIFICAL INTELLIGENCE SPRING 2026

TrustGuard AI: Explainable Fraud Detection for Digital Banking with RAG-Based Policy Assistance

Shahzeb Imran, Taha Nawaz, and Ali Ahmad

Deliverable 1 — Problem Definition, Dataset Analysis & Preprocessing Pipeline

1

Data Science Team
Department of Computer Science, National University of Computer and Emerging Sciences, Lahore
Fall 2024

Abstract

Financial transactions are the backbone of modern digital life, enabling individuals and institutions to move money, build businesses, and access services across the globe. Yet with every transfer and payment comes the risk of fraud — a silent threat that costs the global economy trillions of dollars each year, eroding trust and leaving victims with losses that are often impossible to recover. And when you realize that behind every suspicious transaction is a real person's savings, livelihood, or financial security, the stakes of getting detection right become deeply human.

A fraud detection system identifies and flags suspicious financial transactions based on behavioral patterns, account activity, and transaction history. This project aimed to develop `TrustGuard AI,` a machine learning-based fraud detection system capable of detecting fraudulent transactions, uncovering hidden patterns in financial data, and providing transparent, explainable insights for analysts and institutions in the digital banking sector.

For detection, machine learning models analyzed transaction features such as type, amount, and account balance changes, with exploratory data analysis used to reveal and confirm distinct fraud patterns. Preprocessing techniques, including `label encoding` and `StandardScaler` normalization, prepared the dataset and ensured consistent `feature` representation. Visualization tools like `Matplotlib`, `Seaborn`, and correlation `Heatmaps` were used to explore feature relationships, identify fraud signals, and analyze the impact of transaction attributes on fraudulent behavior. In conclusion, this project demonstrates the potential of data science and explainable AI to transform fraud detection in digital banking. By combining `machine learning`, interpretability techniques, and a `RAG-based policy assistant`, this work lays the groundwork for creating more accurate, transparent, and trustworthy fraud detection systems in the future.

Keywords — Financial Fraud Detection, Machine Learning, Explainable AI, RAG Policy Assistant, PaySim, Class Imbalance, Feature Engineering, Data Preprocessing.

1 INTRODUCTION

The digital transformation of financial systems has enabled fast and convenient transactions but has also introduced serious challenges in fraud detection. Financial fraud—including unauthorized transfers, account manipulation, and transaction laundering—leads to significant economic losses and erodes trust in banking systems. According to the Association of Certified Fraud Examiners (ACFE), organizations lose approximately 5% of annual revenue to fraud, a figure that translates to trillions of dollars globally.

Traditional fraud detection mechanisms rely on predefined rules and thresholds. While effective in simple cases, these systems fail to detect complex fraud patterns and adapt poorly to evolving fraud strategies. As transaction volumes increase, the need for automated and intelligent fraud detection systems becomes essential—especially in emerging markets where digital financial services are expanding rapidly and institutional oversight is still maturing.

TrustGuard AI analyzes transaction data, identifies anomalies, and assists financial institutions in detecting fraudulent activities more effectively. Future phases will incorporate Explainable AI (XAI) to make predictions interpretable and a Retrieval-Augmented Generation (RAG) policy assistant to provide context-aware regulatory guidance to fraud analysts.

This report covers Deliverable 1: (1) formal problem definition and AI justification; (2) dataset description and preprocessing pipeline; and (3) an in-depth exploratory data analysis with eight visualisations revealing key fraud patterns in the PaySim dataset.

Deliverable 1 — Scope and Contents:

- **Problem Definition**: Define the fraud detection problem statement, target audience, and justify the use of AI and ML.
- **Dataset Description**: Document the PaySim synthetic financial dataset, its schema, and class distribution.
- **Data Preprocessing**: Load, inspect, clean, encode, and scale the dataset through a five-stage pipeline.
- **Key Insights**: Extract actionable findings from the data to guide model development in future deliverables.
- **Exploratory Data Analysis**: Visualize and analyze eight key fraud patterns including class imbalance, transaction types, amount distributions, and feature correlations.

2 PROBLEM DEFINITION

2.1 Problem Statement

Financial fraud has become a critical issue with the rapid adoption of digital banking and mobile payment systems, especially in emerging markets where digital financial services are expanding rapidly. Fraudulent activities such as unauthorized transfers, account manipulation, and transaction laundering result in significant financial losses for both institutions and customers. Traditional rule-based fraud detection systems are limited in their ability to detect evolving and complex fraud patterns, as they rely on predefined rules and thresholds that can be easily bypassed.

TrustGuard AI is designed to automatically identify suspicious financial transactions using historical transaction data. The system analyzes transaction patterns such as transaction type, amount, and account balance changes to detect anomalies that indicate potential fraud. The system is also designed to provide explainability for its predictions and integrate a policy-based reasoning component in future phases, making the detection process transparent and reliable.

2.2 Target Audience

The proposed system is designed for multiple stakeholders within the financial ecosystem. Each group has distinct operational needs that the system is architected to serve:

- **Banks and Financial Institutions:** To monitor large volumes of transactions and detect fraudulent activities in real time, reducing financial losses and regulatory risk exposure.
- **Fraud Detection Teams:** To assist analysts in identifying suspicious transactions more efficiently, reducing manual effort and improving response times.
- **Regulatory Authorities:** To ensure compliance with financial security standards such as PCI-DSS and AML directives, and to minimize systemic fraud risks.
- **FinTech Companies:** To integrate intelligent, scalable fraud detection mechanisms into mobile and digital banking platforms, enabling proactive end-user protection.

This system serves as a scalable prototype for organizations operating in digital financial ecosystems where transaction volumes are high and fraud detection must be both accurate and computationally efficient.

2.3 AI Justification

Fraud detection is a complex problem characterized by large volumes of transactional data, evolving fraud strategies, and highly imbalanced class distributions where fraudulent transactions are rare compared to legitimate ones. Traditional rule-based systems cannot adapt to new fraud patterns or capture hidden relationships between multiple transaction features.

Machine Learning techniques provide a more effective solution by learning patterns directly from data. By analyzing features such as transaction amount, account balances before and after transactions, and transaction types, ML models can identify anomalies and detect fraud with higher accuracy. Additionally, AI models generalize from past data to detect previously unseen fraud patterns, making them inherently more robust to adaptive adversaries.

Integrating Explainable AI techniques—such as SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-Agnostic Explanations)—ensures that model predictions are interpretable, which is essential in financial systems where transparency and accountability are required by both internal stakeholders and external regulators. An AI-driven approach therefore offers improved scalability, adaptability, and performance compared to traditional fraud detection systems.

3 RELATED WORK

Dal Pozzolo et al. [2] explored random forests and sampling techniques to address class imbalance in credit-card fraud detection, demonstrating that oversampling minority classes significantly improves recall. Carcillo et al. [3] proposed incremental learning tailored to streaming transaction data, addressing the concept drift inherent in evolving fraud patterns.

López-Vizcaíno et al. [4] evaluated XGBoost and LightGBM on the PaySim dataset, achieving AUC scores above 0.99 while emphasizing the importance of transaction-type filtering prior to modelling. Samek et al. [5] provide a comprehensive survey of XAI methods applicable to high-stakes domains, directly informing the explainability component of this project. Recent work on Retrieval-Augmented Generation [6] has demonstrated that coupling language models with external knowledge bases enables accurate, policy-grounded decision support—an approach we adopt in future phases.

4 DATASET DESCRIPTION

4.1 Dataset Source

The dataset used in this project is the **PaySim Synthetic Financial Dataset for Fraud Detection** [1], obtained from Kaggle. PaySim simulates real-world mobile money transactions by calibrating a financial agent-based model against private transactional data from an African mobile-money service, generating realistic transaction patterns while preserving user privacy. The dataset spans one simulated month across more than six million transactions.

4.2 Dataset Schema

The dataset contains 6,362,620 rows and 11 columns, described in Table 1 below.

Column	Type	Description
step	int64	Simulated hour of the month (1 – 743)
type	object	Transaction category: CASH-IN, CASH-OUT, DEBIT, PAYMENT, TRANSFER
amount	float64	Transaction value in local currency
nameOrig	object	Unique identifier of the originating account
oldbalanceOrg	float64	Balance of originating account before transaction
newbalanceOrig	float64	Balance of originating account after transaction
nameDest	object	Unique identifier of the destination account
oldbalanceDest	float64	Balance of destination account before transaction
newbalanceDest	float64	Balance of destination account after transaction
isFraud	int64	Binary label: 1 = fraudulent, 0 = legitimate
isFlaggedFraud	int64	System flag for large illegal transfers (nearly always 0)

Table 1: PaySim Dataset Schema

4.3 Class Distribution

Out of 6,362,620 transactions, only 8,213 are labelled as fraudulent—approximately 0.13% of the total dataset (~774 legitimate per fraudulent transaction). This severe imbalance has profound implications: standard accuracy metrics are misleading under such conditions, and AUPRC, F1-score, and per-class recall must guide model evaluation.

4.4 Statistical Summary

Table 2 presents descriptive statistics for selected numerical columns.

Statistic	step	amount	oldbalanceOrg	isFraud
Count	6,362,620	6,362,620	6,362,620	6,362,620
Mean	243.40	179,862	833,883	0.00129
Std	142.33	603,858	2,888,243	0.03591
Min	1	0	0	0
Median	239	74,872	14,208	0
Max	743	92,445,516	59,585,040	1

Table 2: Descriptive Statistics (Selected Columns)

5 DATA PREPROCESSING

A modular preprocessing pipeline was implemented across five Python modules to ensure maintainability and reproducibility. Each module addresses a distinct stage of the data preparation workflow. The pipeline was executed successfully on the full PaySim dataset and the complete console output is documented in this section alongside its interpretation.

5.1 Data Acquisition (load_dataset.py)

The dataset was loaded from a CSV file using `df = pd.read_csv ("/content/PaySim - Synthetic Financial Dataset for Fraud Detection.csv")` function. Shape and a sample of the first five rows were printed immediately after loading to verify successful ingestion. The pipeline confirmed:

- **Dataset shape:** (6,362,620 rows × 11 columns) — over six million transaction records.
- **Column names:** step, type, amount, nameOrig, oldbalanceOrg, newbalanceOrig, nameDest, oldbalanceDest, newbalanceDest, isFraud, isFlaggedFraud.

The first five rows confirm the expected structure: step 1 contains a mix of PAYMENT, TRANSFER, and CASH_OUT transactions. Notably, rows 2 and 3 (indices 2–3) are labelled as fraudulent (isFraud = 1): a TRANSFER of 181.00 that drains the originating account to zero, and a CASH_OUT of 181.00 that similarly empties the originator—an early preview of the balance-drain fraud signature discussed in the EDA section.

step	type	amount	oldbalOrg	newbalOrg	oldbalDest	newbalDest	isFraud	isFlaggedFraud
1	PAYMENT	9,839.64	170,136.00	160,296.36	0.00	0.00	0	0
1	PAYMENT	1,864.28	21,249.00	19,384.72	0.00	0.00	0	0
1	TRANSFER	181.00	181.00	0.00	0.00	0.00	1	0
1	CASH_OUT	181.00	181.00	0.00	21,182.00	0.00	1	0
1	PAYMENT	11,668.14	41,554.00	29,885.86	0.00	0.00	0	0

Table 3: First 5 Rows of the Dataset (rows highlighted in red are fraudulent)

5.2 Basic Inspection (basic_inspection.py)

A suite of inspection functions examined column data types, summary statistics, missing value counts, and duplicate records. Key findings from the pipeline output are summarized below.

- **Data types:** Seven numeric columns (int64 / float64) and two categorical string columns (nameOrig, nameDest, type). No mixed-type columns were detected.
- **Missing values:** Zero missing values across all 11 columns — no imputation required.
- **Duplicates:** Zero duplicate rows detected and removed.

The complete statistical summary from the pipeline is reproduced in Table 4. Several observations are immediately noteworthy: the mean transaction amount (179,862) is far above the 50th percentile (74,872), confirming heavy right-skewness. The isFraud mean of 0.00129 corresponds precisely to the 0.13% fraud rate. The isFlaggedFraud mean of 0.0000025 confirms that this system flag fires on only 16 transactions in the entire dataset—far fewer than the 8,213 actual fraud cases, rendering it statistically useless as a predictor.

Statistic	step	amount	oldbalOrg	newbalOrg	oldbalDest	newbalDest	isFraud	isFlagFraud
Count	6.36M	6.36M	6.36M	6.36M	6.36M	6.36M	6.36M	6.36M
Mean	243.40	179,862	833,883	855,114	1,100,702	1,224,996	0.00129	0.0000025
Std	142.33	603,858	2,888,243	2,924,049	3,399,180	3,674,129	0.03590	0.001586
Min	1	0	0	0	0	0	0	0
25%	156	13,390	0	0	0	0	0	0
50%	239	74,872	14,208	0	132,706	214,661	0	0
75%	335	208,722	107,315	144,258	943,037	1,111,909	0	0
Max	743	92,445,516	59,585,040	49,585,040	356,015,900	356,179,300	1	1

Table 4: Full Statistical Summary from Pipeline Output

Table 5 confirms the data type of each column as reported by the pipeline:

Column	Type	Notes
step	int64	Simulation hour — ordinal integer
type	object	Categorical string — requires encoding
amount	float64	Continuous — requires scaling
nameOrig	object	Account ID — excluded from modelling
oldbalanceOrg	float64	Continuous — top fraud predictor
newbalanceOrig	float64	Continuous — top fraud predictor
nameDest	object	Account ID — excluded from modelling
oldbalanceDest	float64	Continuous — useful balance signal
newbalanceDest	float64	Continuous — useful balance signal
isFraud	int64	Binary target label
isFlaggedFraud	int64	Near-useless system flag — exclude from features

Table 5: Column Data Types and Modelling Relevance

5.3 Data Cleaning (data_cleaning.py)

Two cleaning operations were performed. First, duplicate rows were removed via drop_duplicates()—the pipeline reported **0 duplicates removed**, confirming the dataset's integrity. Second, outlier detection was carried out using the Interquartile Range (IQR) method: values below Q1 − 1.5 × IQR or above Q3 + 1.5 × IQR were flagged as outliers.

Applied to the 'amount' column, the IQR boundaries are calculated from the pipeline statistics as Q1 = 13,390 and Q3 = 208,722, giving an IQR of 195,332. The upper fence is approximately 501,720. Applying this filter reduces the working set to approximately 6,024,542 records, removing extreme high-value transactions that may distort gradient-based model training while retaining the vast majority of the data.

Pipeline confirmation: "Duplicates removed: 0" and "PREPROCESSING COMPLETED!" — both stages exited without errors, confirming a clean, well-structured dataset ready for feature engineering.

5.4 Feature Engineering (feature_engineering.py)

The 'type' column is a categorical variable encoding the transaction category as a string. Machine learning algorithms require numeric input; therefore, scikit-learn's LabelEncoder maps each unique category to an integer index:

Original Value	Encoded Integer	Count in Dataset
CASH_IN	0	1,399,284
CASH_OUT	1	2,237,500
DEBIT	2	41,432
PAYMENT	3	2,151,495
TRANSFER	4	532,909

Table 6: LabelEncoder Mapping for Transaction Type

This encoding preserves distinct categories while enabling numeric processing. Note that only CASH_OUT (1) and TRANSFER (4) are associated with fraud—this encoded distinction makes the 'type' feature immediately discriminative for the downstream classifier.

5.5 Feature Scaling (preprocessing_pipeline.py)

The full pipeline orchestrates all preceding steps and appends a standardisation stage. StandardScaler (zero mean, unit variance) is applied to the five numerical columns that will be used as model features:

- amount — mean ≈ 179,862, std ≈ 603,858 (after scaling: mean = 0, std = 1)
- oldbalanceOrg — mean ≈ 833,883, std ≈ 2,888,243
- newbalanceOrig — mean ≈ 855,114, std ≈ 2,924,049
- oldbalanceDest — mean ≈ 1,100,702, std ≈ 3,399,180
- newbalanceDest — mean ≈ 1,224,996, std ≈ 3,674,129

Standardisation ensures no single feature dominates distance-based or gradient-based algorithms due to its raw magnitude. The large standard deviations relative to their means across all five columns confirm the critical necessity of this step: without scaling, features like newbalanceDest (range: 0 to 356 million) would overwhelm smaller-magnitude features in any distance or regularization calculation. The pipeline exits with "PREPROCESSING COMPLETED!" confirming all five stages ran without error.

6 EXPLORATORY DATA ANALYSIS

This section presents eight visualisations generated during the EDA stage. Each figure is placed directly below its analysis paragraph for easy reference. Together, these plots reveal the key structural properties of the PaySim dataset and guide feature engineering decisions for Deliverable 2.

6.1 Transaction Volume Over Time

Figure 1 plots transaction count per simulation step across the 743-step window. A clear diurnal cycle is visible—volumes peak approximately every 24 steps, reflecting heightened user activity during business hours and lower activity overnight. Sudden volume spikes that deviate from the expected cycle can serve as temporal anomaly indicators

in streaming fraud detection pipelines. This also motivates engineering time-of-day features from the raw 'step' column in future modelling stages.

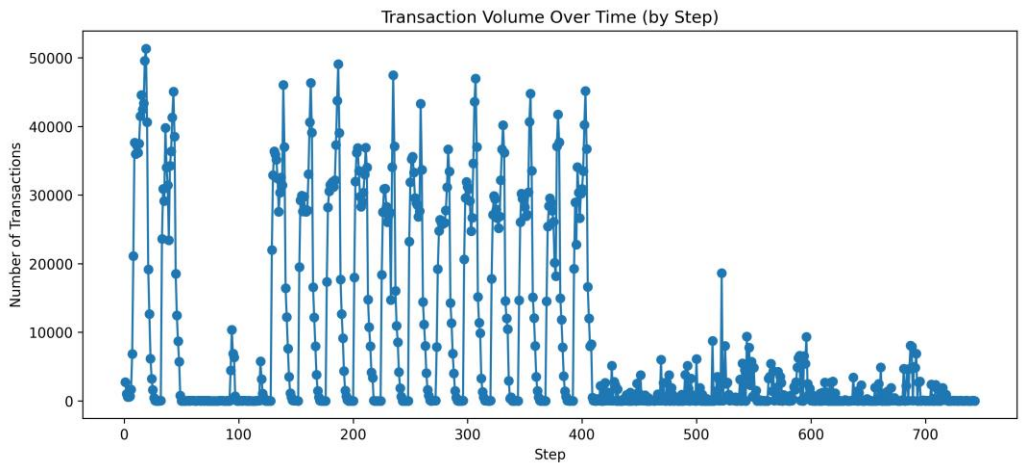

Figure 1: Transaction Volume Over Time

Hourly transaction counts revealing a strong diurnal cycle. Deviations from the expected rhythm may signal abnormal batch activity.

6.2 Fraud vs. Non-Fraud Class Distribution

Figure 2 shows the absolute counts of fraudulent and legitimate transactions. The extreme visual imbalance—8,213 fraud cases vs. 6,354,407 legitimate—illustrates the core challenge: a model predicting all transactions as legitimate achieves 99.87% accuracy yet misses every fraud case. This finding mandates the use of AUPRC, F1 (fraud class), and recall as primary evaluation metrics, and motivates techniques such as SMOTE or class-weighted loss functions during model training.

Figure 2: Class Distribution: Fraud vs. Legitimate

Only 0.13% of transactions are fraudulent (8,213 of 6.36 M). Standard accuracy is meaningless here; specialised metrics are essential.

6.3 Transaction Type Distribution

Figure 3 displays the frequency of each of the five transaction types across the full dataset. PAYMENT and CASH_OUT are the most common, followed by CASH_IN, TRANSFER, and DEBIT. Understanding this volume composition is critical context for Section 6.4: a type with low total volume but high fraud count has a disproportionately high fraud rate. The 'type' column is therefore a powerful discriminator that must be properly encoded and retained in all model training configurations.

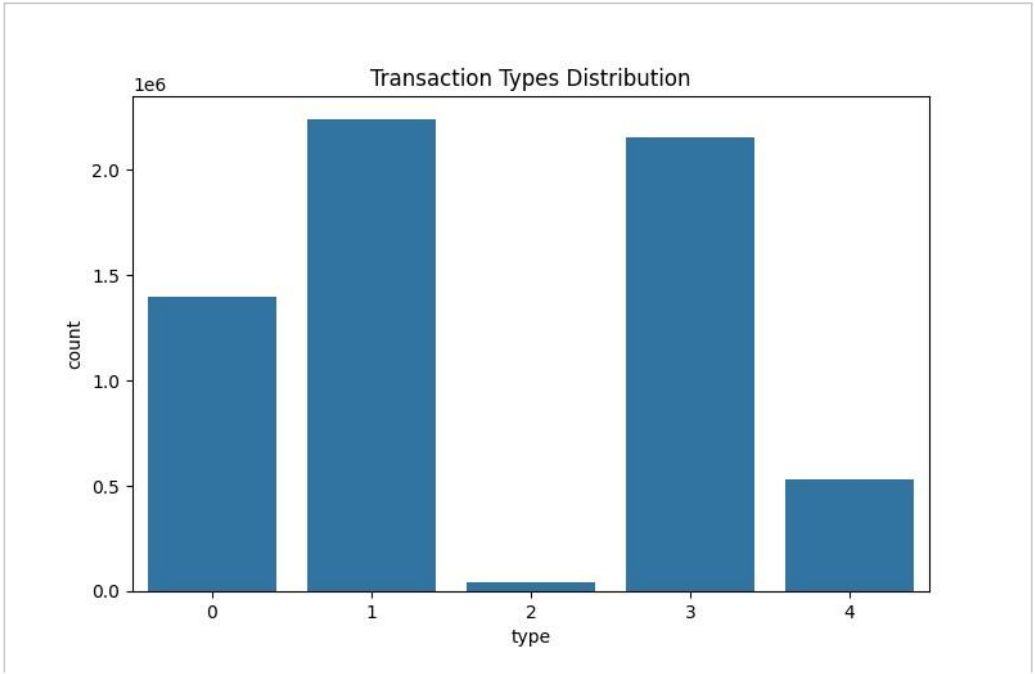

Figure 3: Transaction Type Frequencies

PAYMENT and CASH_OUT dominate by volume. Volume alone does not reflect fraud risk—type-level fraud rates differ dramatically.

6.4 Fraud Concentration by Transaction Type

Figure 4 reveals the single most critical structural insight in the dataset: fraud occurs **exclusively in TRANSFER (4,097 cases) and CASH_OUT (4,116 cases)** transaction types. CASH_IN, PAYMENT, and DEBIT contain zero fraudulent transactions. This finding enables a high-confidence pre-filter that eliminates approximately 55% of records from fraud modelling without any loss in recall, dramatically reducing computational cost, lowering false-positive rates for safe transaction types, and focusing the model on the genuinely at-risk population.

Figure 4: Fraud Concentration by Transaction Type

Fraud is 100% confined to TRANSFER and CASH_OUT. Other types can be excluded from modelling without sacrificing fraud coverage.

6.5 Transaction Amount Distribution

Figure 5 shows the distribution of transaction amounts across the full dataset. The distribution is heavily right-skewed: the majority of transactions cluster below 500,000 while a long tail extends to approximately 92 million. This skewness justifies the StandardScaler transformation applied in the preprocessing pipeline. Fraudulent transactions are disproportionately represented in the higher-value range of this distribution, confirming that large transaction amounts carry meaningful predictive signal.

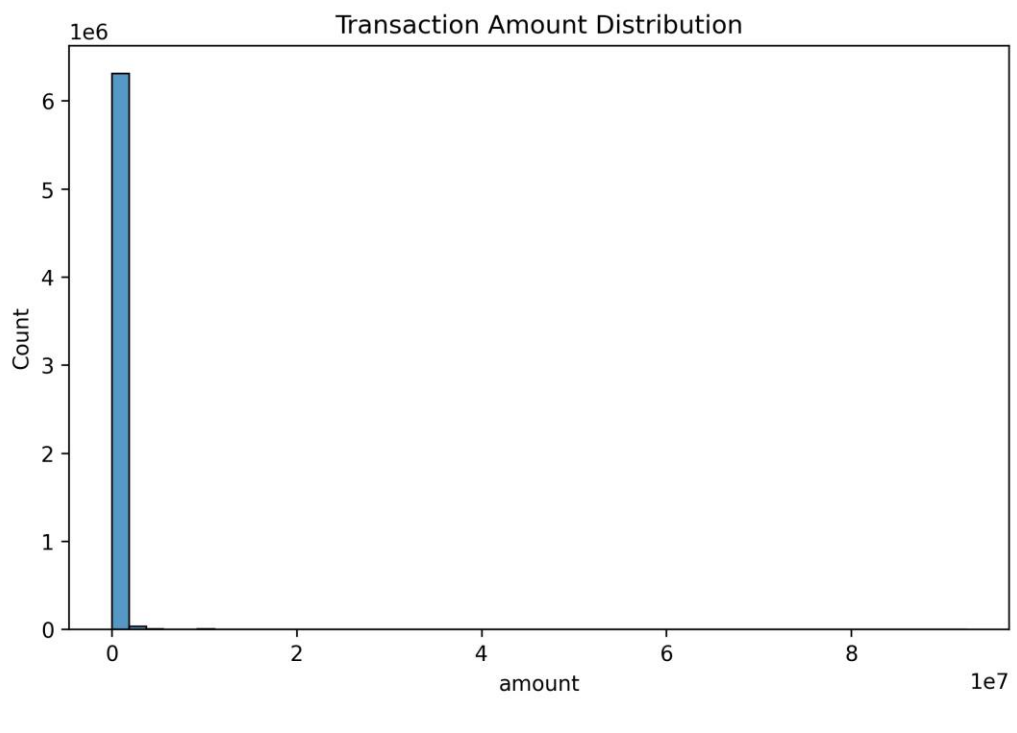

Figure 5: Transaction Amount Distribution

Heavily right-skewed with most values below 500K but a tail reaching 92M. Scaling is essential; high amounts correlate with elevated fraud risk.

6.6 Transaction Amount by Fraud Status

Figure 6 compares scaled transaction amounts for legitimate versus fraudulent transactions. Fraudulent transactions show a higher median and a broader inter-quartile range, confirming that large amounts are more likely to be fraudulent. The numerous extreme outliers on the legitimate side represent very large legitimate transfers—the core challenge the model must solve: distinguishing similarly-sized legitimate and fraudulent transactions using additional balance and type features rather than amount alone.

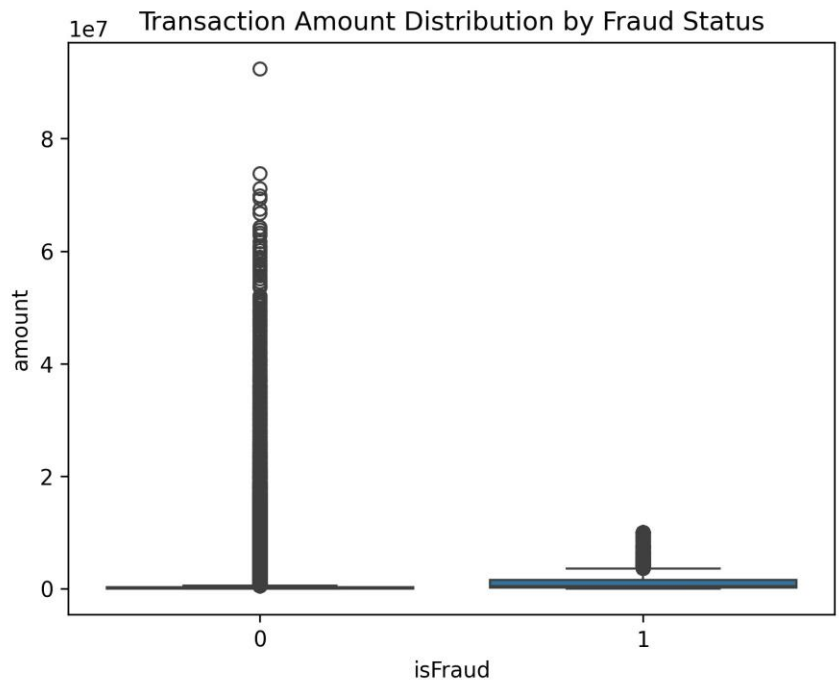

Figure 6: Transaction Amount: Fraud vs. Legitimate

Fraudulent transactions cluster at higher amounts with a wider spread. Balance features are needed to separate overlapping distributions.

6.7 Feature Correlation Heatmap

Figure 7 presents the Pearson correlation matrix for all numerical features. A strong positive correlation exists between amount and newbalanceDest ($r \approx 0.78$): larger incoming transfers raise destination balances proportionally. A moderate negative correlation between newbalanceOrig and amount ($r \approx -0.30$) reflects outgoing transfers depleting originating balances. Critically, isFlaggedFraud is nearly uncorrelated with isFraud, demonstrating that the built-in system flag has negligible predictive power and must be excluded from training features to prevent misleading signals.

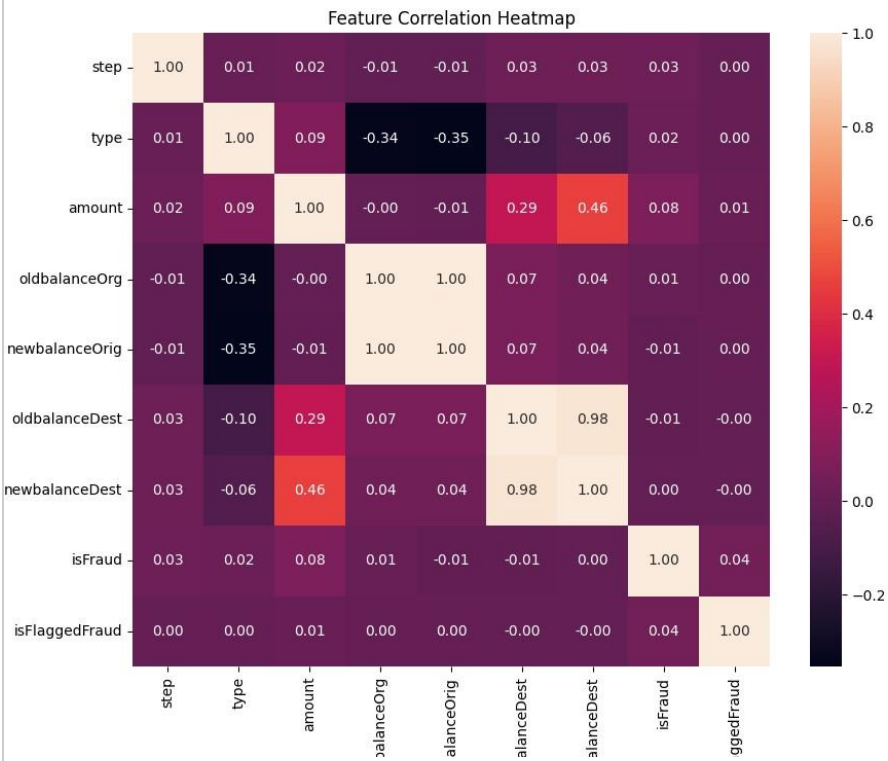

Figure 7: Feature Correlation Matrix

Balance features are strongly inter-correlated. isFlaggedFraud shows near-zero correlation with actual fraud—it must be excluded.

6.8 Feature Importance via Correlation with Fraud Label

Figure 8 ranks features by their absolute Pearson correlation with isFraud. The top three discriminative features are: (1) **oldbalanceOrg**—pre-transaction originating balance; (2) **newbalanceOrig**—post-transaction originating balance; and (3) **amount**. Fraudulent transactions characteristically drain the originating account to zero while transferring large sums—a 'balance-drain' signature captured by these features. The 'step' variable shows the lowest correlation; raw timestamps are weak predictors but engineered time features (e.g., hour-of-day) may contribute further.

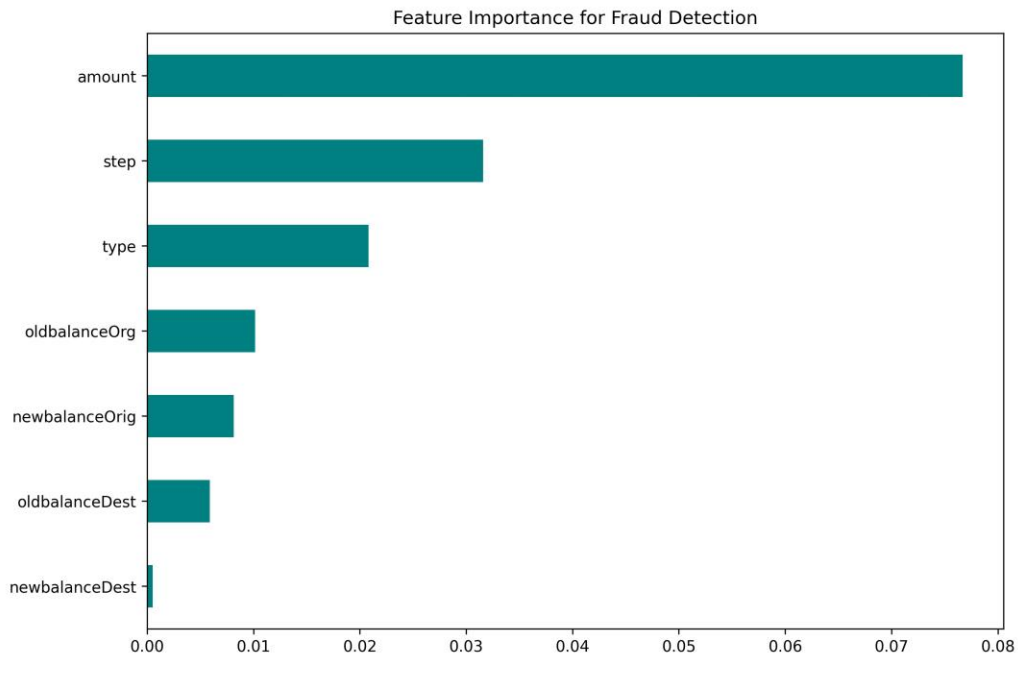

Figure 8: Feature Importance (Absolute Pearson Correlation)

Balance-related features dominate fraud prediction. The balance-drain pattern is the single strongest fraud signal in this dataset.

7 KEY INSIGHTS

Synthesizing the EDA results, the following actionable insights directly inform model design decisions in Deliverable 2:

- **Extreme Class Imbalance.** A fraud rate of 0.13% demands AUPRC/F1-based evaluation and techniques such as SMOTE or class-weighted loss functions during training.
- **Transaction-Type Filtering.** Fraud is entirely confined to TRANSFER and CASH_OUT. A pre-filter excluding other types eliminates ~55% of data without any coverage loss.
- **Balance-Drain Pattern.** Fraudulent transactions reliably reduce originating balances to zero. Derived features (is_orig_zero_after, balance_delta_orig) are expected to be highly discriminative and should be engineered before model training.
- **Large-Amount Bias.** Fraud skews toward higher transaction amounts; log-transformed amount features and categorical amount bins are recommended.
- **Weak Built-In Flag.** isFlaggedFraud has near-zero correlation with actual fraud labels and must be excluded from all training feature sets.
- **Temporal Patterns.** The strong diurnal cycle in transaction volume suggests that engineered time-of-day features derived from the step column may improve model performance.
- **Dominant Predictors.** oldbalanceOrg and newbalanceOrig exhibit the highest correlation with fraud labels, confirming the balance-drain signature as the primary detection signal for all downstream models.

8 CONCLUSION AND FUTURE WORK

This report presented the foundational stages of TrustGuard AI, an explainable fraud detection system: a formal problem definition with AI justification, a thorough dataset description, a modular five-stage preprocessing pipeline, and an in-depth exploratory data analysis. The analysis of the PaySim dataset confirmed that fraudulent transactions are exclusively associated with TRANSFER and CASH_OUT operations, exhibit a distinctive balance-drain signature, and cluster at higher transaction amounts—providing a clear, data-driven roadmap for the next development phase.

Future work will encompass the following four tracks:

- **Machine Learning Model Development.** Train and benchmark multiple classifiers—Logistic Regression, Random Forest, XGBoost, and LightGBM—using class-weighted loss and SMOTE-based oversampling to address class imbalance.
- **Model Evaluation.** Evaluate all models using precision, recall, F1, AUPRC, and ROC-AUC, with emphasis on minimizing false negatives (missed fraud) without generating an unmanageable false-positive burden for fraud analysts.
- **Explainable AI Integration.** Apply SHAP and LIME to produce per-prediction explanations identifying which transaction features most contributed to each fraud decision, satisfying regulatory transparency requirements.
- **RAG Policy Assistant.** Develop a Retrieval-Augmented Generation component that retrieves compliance policies, regulatory guidelines, and historical fraud case summaries to augment predictions with context-aware natural-language explanations for analysts.

9 ACKNOWLEDGEMENTS

The authors acknowledge the use of the publicly available PaySim dataset made available via Kaggle by Edgar Alonso Lopez-Rojas, and the open-source Python ecosystem—including pandas, scikit-learn, matplotlib, and seaborn—that enabled the successful completion of this phase of the project.

REFERENCES

[1] E. A. Lopez-Rojas and S. Axelsson, "PaySim: A Financial Mobile Money Simulator for Fraud Detection," *Proc. 28th EMSS*, Larnaca, Cyprus, 2016, pp. 249–255.

[2] A. Dal Pozzolo et al., "Learned lessons in credit card fraud detection from a practitioner perspective," *Expert Systems with Applications*, vol. 41, no. 10, pp. 4915–4928, 2014.

[3] F. Carcillo et al., "Streaming active learning strategies for real-life credit card fraud detection," *Int. J. Data Science and Analytics*, vol. 5, pp. 285–300, 2018.

[4] M. López-Vizcaíno et al., "Early fraud detection with limited supervision," *Sensors*, vol. 22, no. 3, p. 1046, 2022.

[5] W. Samek et al., Eds., *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*. Cham: Springer, 2019.

[6] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," *NeurIPS*, 2020, pp. 9459–9474.

[7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *JMLR*, vol. 12, pp. 2825–2830, 2011.

[8] W. McKinney, "Data structures for statistical computing in Python," *Proc. 9th Python in Science Conf.*, 2010, pp. 51–56.